

LAB 3

High-Definition Multimedia Interface — HDMI

Name: Ioannis Papadopoulos, Maria Ilektra Tzormpatzaki

Student ID: 03899, 03993

Date: 19/12/2025

Board: Boolean Board (Spartan-7 FPGA)

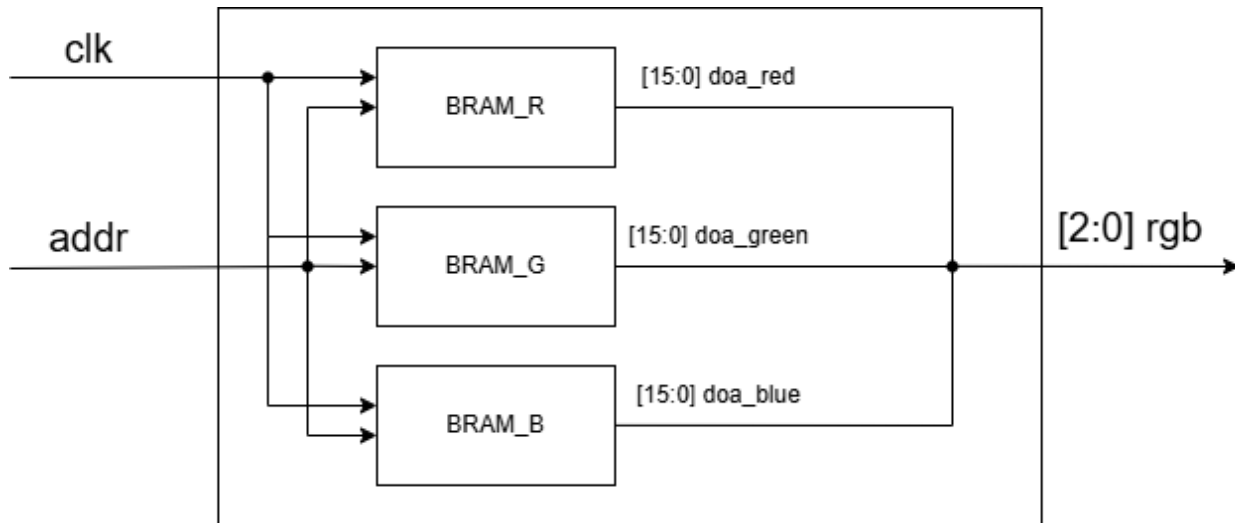
Contents

1. Part A – Video RAM.....	2
1.1 Introduction	2
1.2 Implementation	2
1.3 Verification	3
2. Part B – Pixel Counters, Synchronization Signals, and VRAM Addressing	4
2.1 Introduction	4
2.2 Implementation	4
2.3 Verification	5
3. Part C – TMDS Encoding.....	8
3.1 Introduction	8
3.2 Implementation	8
3.3 Verification	9
4. Part D – HDMI Controller Integration.....	10
4.1 Introduction	10
4.2 Implementation	10
4.3 Verification	11
4.4 Experiment / Final Implementation.....	11
5. Part D EXTRA – HDMI Controller Integration.....	12
5.1 Introduction/Implementation	12
5.2 Verification	12
5.4 Experiment / Final Implementation.....	13

1. Part A – Video RAM

1.1 Introduction

In Part A, a simple Video RAM (VRAM) was implemented to store the color information of the image displayed by the HDMI controller. The VRAM is responsible for mapping each pixel address to a corresponding RGB color value, using one bit per color channel, thus supporting a total of eight distinct colors.



1.2 Implementation

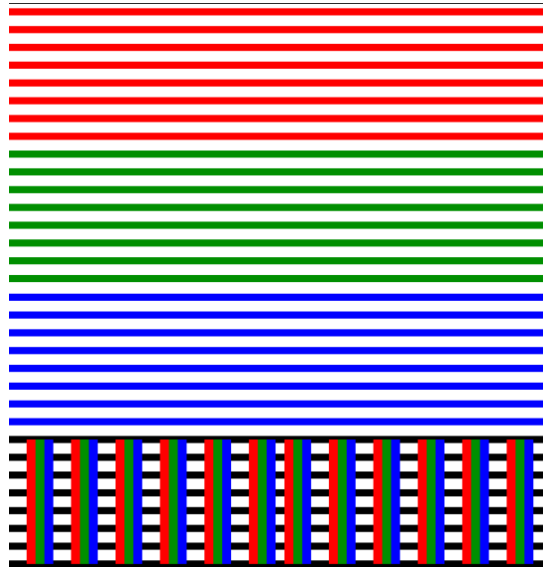
The VRAM architecture is based on the use of three independent Block RAM (BRAM) blocks, one for each primary color channel: **Red**, **Green**, and **Blue**. Each BRAM stores a single-bit value per pixel, indicating whether the corresponding color component is active.

Xilinx RAMB18E1 primitives were instantiated directly. This approach ensures that the memory is explicitly mapped to the FPGA's internal BRAM resources. Each RAMB18E1 block is configured to operate with a 1-bit data width, which matches the requirements of the color channels. Only Port A of each BRAM is used and configured for read-only operation, while Port B is disabled.

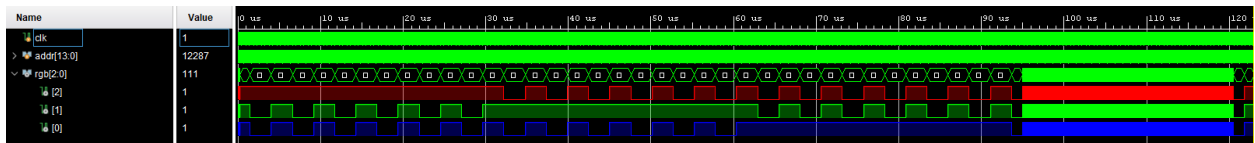
A single shared address bus is connected to all three BRAM instances. This guarantees that the same pixel address is used simultaneously for the **Red**, **Green**, and **Blue** memories, so that each address uniquely corresponds to one RGB color value. The outputs of the BRAMs are 1-bit values, which are extracted from the least significant bit of each BRAM output word. Finally, the three single-bit color outputs are combined to form a 3-bit RGB signal, where each bit represents one color channel. This RGB output is then forwarded to the subsequent stages of the HDMI controller for further processing and display.

1.3 Verification

The test bench increments the address by one at every positive edge of the clock, until every pixel of the image is passed. The image instantiated in this verification is the one that is recommended in the assignment:

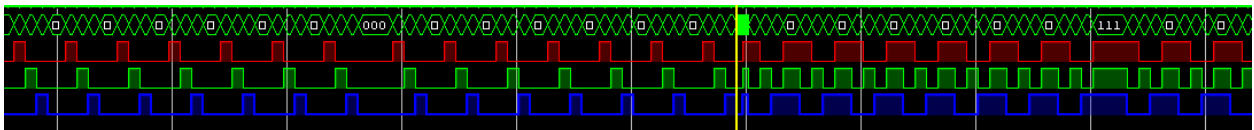


1. The full waveform:



In the first part, 0-95us, is where the horizontal lines will appear. At the start, `rgb[2]` is always high and the other colors change repeatedly. `rgb[1]` and `rgb[0]` follow with the same logic. Pixels that have all their colors on high will be shown as white.

2. The column part of the picture.



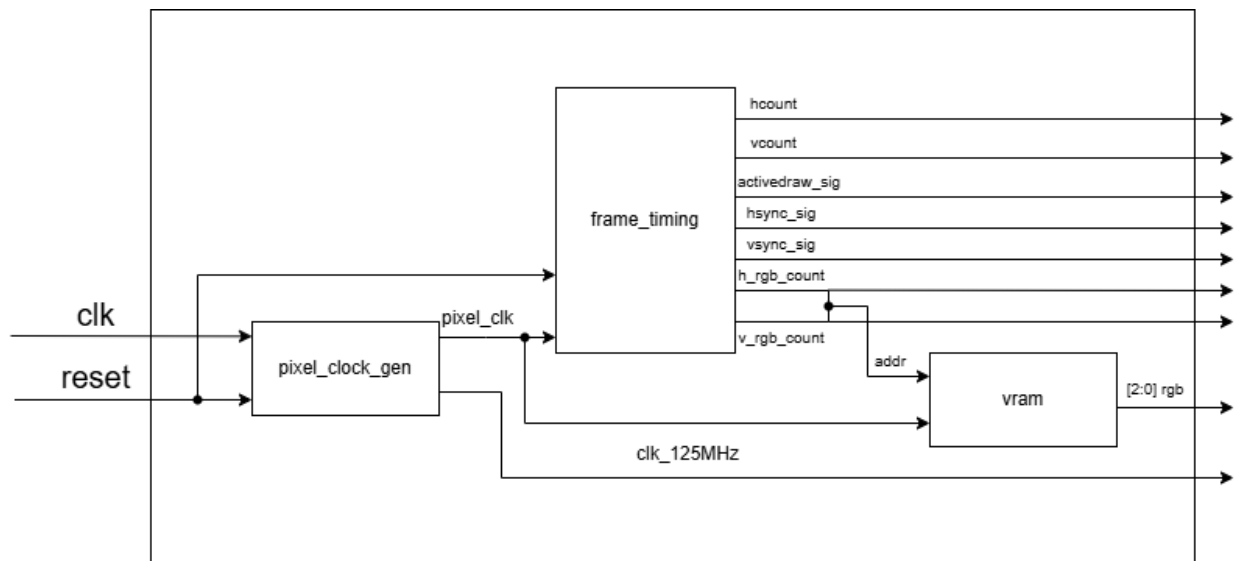
In the first half of the waveform, the pattern corresponds to a column displaying the three primary colors against a black background. In the second half of the waveform, the same pattern is shown but with a white background. The black background is produced when `rgb = 3'b000`, while the white background corresponds to `rgb = 3'b111`.

The warnings that appear in Vivado are nothing important. Warnings that appear are the timescale warning and the constrains for write warning. Also the warnings for not instatiating all the connections of the RAMB18E1, which is not necessary to do if they are supposed to stay unused

2. Part B – Pixel Counters, Synchronization Signals, and VRAM Addressing

2.1 Introduction

Part B focuses on the generation of the video timing signals required for a VGA 640×480 display and on the correct addressing of the VRAM implemented in Part A. This part provides the necessary clocking, synchronization, and pixel-position logic that allows the system to traverse the video frame and fetch the appropriate color data for each displayed pixel.



2.2 Implementation

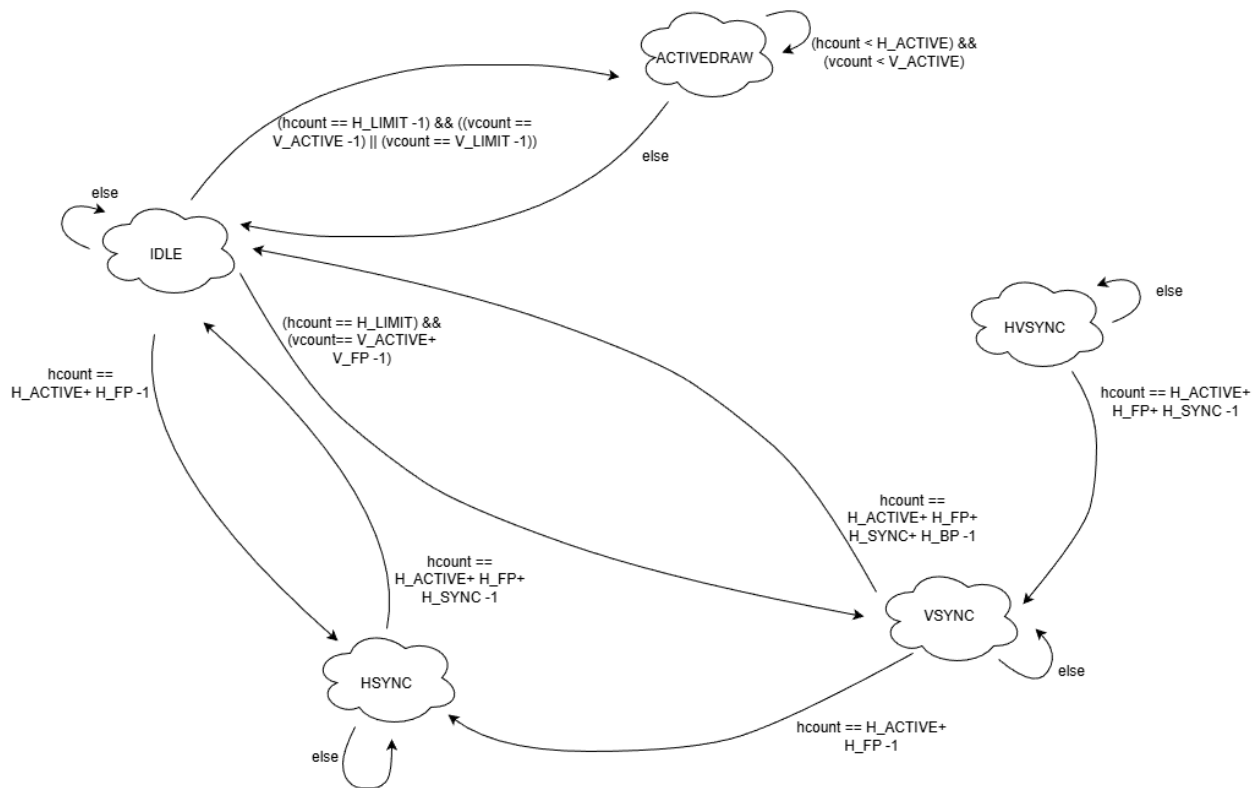
Clock generation is handled using the Mixed-Mode Clock Manager (MMCM) primitive. Starting from the 100 MHz system clock provided by the FPGA, the MMCM generates a pixel clock of approximately 25.175 MHz. In addition, a second clock running five times the pixel clock is generated for use in later stages of the system.

The generation of synchronization signals and pixel position counters is performed by the `frame_timing` module. This module produces horizontal synchronization (`hsync_sig`), vertical synchronization (`vsync_sig`), and active drawing (`activedraw_sig`) signals based on two counters, `hcount` and `vcount`, which iterate over the horizontal and vertical dimensions of the video frame. These counters traverse both the visible display area and the non-visible regions.

Extra counters are introduced, `h_rgb_count` and `v_rgb_count`. The VRAM stores an image of 128×96 pixels, which is expanded spatially so that each stored pixel corresponds to multiple display pixels. The reduced counters increase only during the active drawing region and advance 5 times slower than the main counters, effectively performing pixel replication. This is

implemented with the use of temp_counters, that simultaneously with hcount and vcount, but when they reach the number 4, they reset back to 0. The slower counters increase only when the temp counters have reached the number 4. In this way, hcount and vcount determine the precise display timing, while h_rgb_count and v_rgb_count determine which VRAM pixel is currently being accessed.

The control of timing signals is organized using a Moore FSM inside the frame_timing module. The FSM transitions between states corresponding to active drawing, horizontal synchronization, vertical synchronization, combined synchronization and idle regions.



Finally, the VRAM addressing is performed by combining the reduced-resolution counters into a single address. Specifically, the address is formed as $\{v_rgb_count, h_rgb_count\}$, mapping the two-dimensional pixel position to a unique memory location. This address is given to the module in Part A, that provides the rgb combination.

2.3 Verification

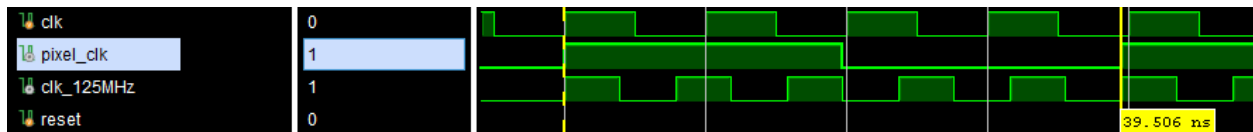
The verification process for the focuses on confirming that all signals, internal state transitions, counters, and output waveforms are correct. To achieve this, simulation is inspected in several focused segments.



For simulation purposes the bounds were made smaller.

- parameter H_LIMIT = 40, V_LIMIT = 30;
- parameter H_ACTIVE = 20, V_ACTIVE = 16;
- parameter H_FP = 6, V_FP = 8;
- parameter H_SYNC = 10, V_SYNC = 2;
- parameter H_BP = 4, V_BP = 4;

1. Verification of the new clocks:



The pixel that is generated has a frequency = $1/39.506 = 25.31 \dots$ Mhz. Although it is not exactly 25.175Mhz it doesn't cause any problems in the execution of PartD. On the other hand the second new clock is exactly 5 times faster than the pixel_clk.

2. Verification of the counters

The horizontal counters:



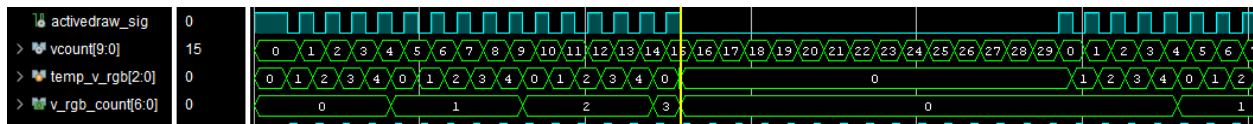
The hcount and temp horizontal counter increases in every positive edge of the pixel clock. As explained before when the temp horizontal counter resets again to 0 then slower horizontal counter increases, effectively making it 5 times slower than the hcount. Also, FSM is not in the activedraw region the temp horizontal counter and the slower counter correctly reset to 0.

The vcount:



The vcount increases after the hcount reaches the end of the line.

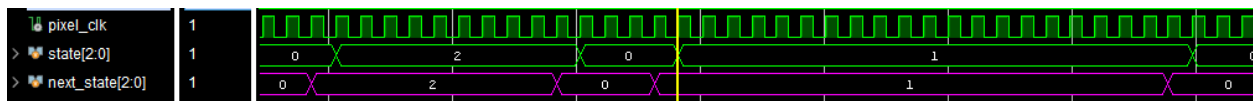
The vertical counters:



The temp vertical counter increments every time the FSM leaves the activedraw state. The reason it doesn't increment the same time as vcount, is because vcount increments when the whole line end (visible and non-visible frame), whereas as the purpose of other 2 vertical counters is to keep track of which pixel is to be given in the PartA, therefore they change according the activedraw signal.

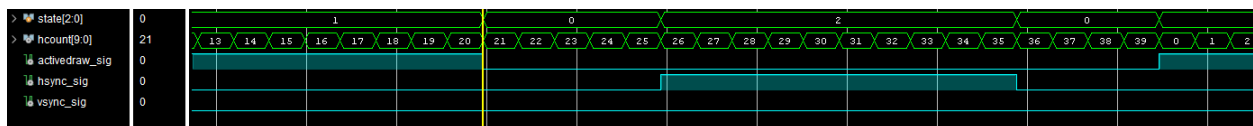
3. Verification of signals and states:

Firstly, the current state's value is updated as intended in the next positive edge of the pixel clock.

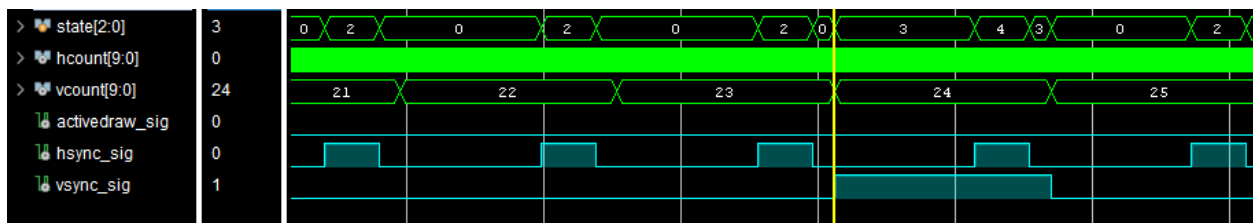


The next three waveforms show that the activedraw, hsync vsync signals are raised the appropriate time relative to the counters. Also, the corresponding current state can be seen in the waveforms.

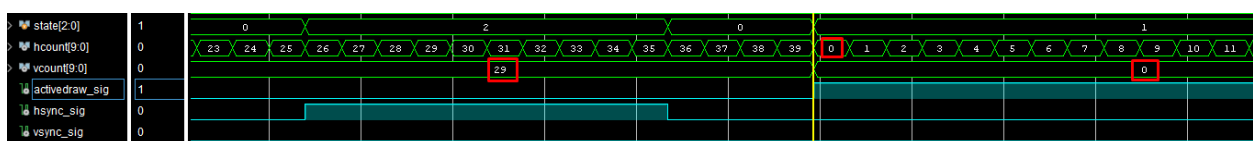
ACTIVEDRAW → IDLE → HSYNC → IDLE → ACTIVEDRAW



IDLE → HSYNC → IDLE → VSYNC → HVSYNC → VSYNC → IDLE



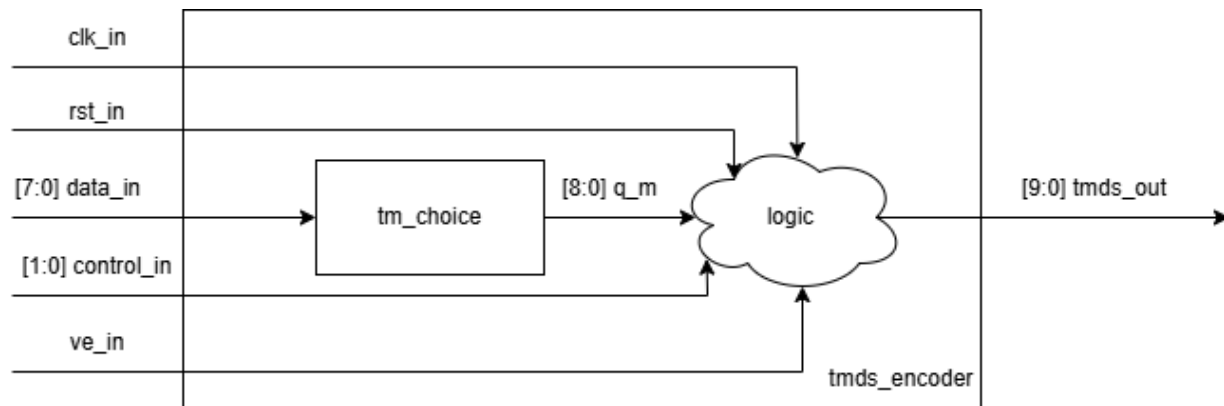
IDLE → HSYNC → IDLE → ACTIVEDRAW



3. Part C – TMDS Encoding

3.1 Introduction

Part C implements the TMDS (Transition Minimized Differential Signaling) encoding algorithm used in HDMI transmission. The objective of this part is to transform parallel pixel and control data into 10-bit TMDS symbols suitable for high-speed serial transmission, strictly following the rules and flowcharts provided in the assignment description.



3.2 Implementation

The implementation is divided conceptually into two main stages: transition minimization and completion of TMDS encoding.

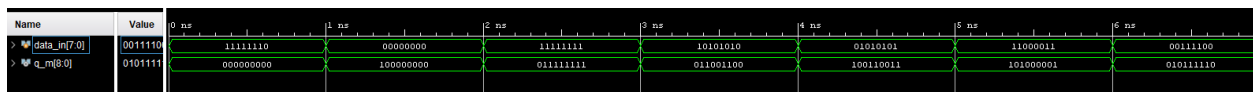
The encoding process begins with the Transition Minimization (TM) stage. In this stage, an 8-bit parallel pixel value is processed to produce a 9-bit intermediate word “q_m”. The algorithm selects between XOR-based or XNOR-based encoding according to the criteria defined in the assignment, which depend on the number of logical ones in the input word and the value of its least significant bit. The primary goal of this stage is to minimize the number of bit transitions between consecutive bits in the encoded word, thereby reducing switching activity. This stage is implemented as a purely combinational block and directly follows the provided flowchart.

The second stage is the TMDS encoder, which accepts the 9-bit “q_m” value and produces the final 10-bit TMDS output word. This stage operates synchronously with the pixel clock and is responsible for ensuring DC balance in the transmitted data stream. To achieve this, the encoder maintains an internal signed tally register that tracks the running disparity between transmitted ones and zeros. Based on the current tally value and the distribution of ones and zeros in “q_m”, the encoder decides whether to invert the data bits and how to set the two most significant bits of the TMDS word. All inversion and selection decisions follow the exact rules specified in the assignment flowcharts.

Special handling is implemented for video enable and control symbols. When the video enable signal “ve_in” is asserted, the encoder processes pixel data through the TMDS pipeline. When ve_in is deasserted, indicating blanking intervals, the encoder outputs one of four predefined TMDS control symbols. The specific control symbol transmitted is selected by the control_in signal.

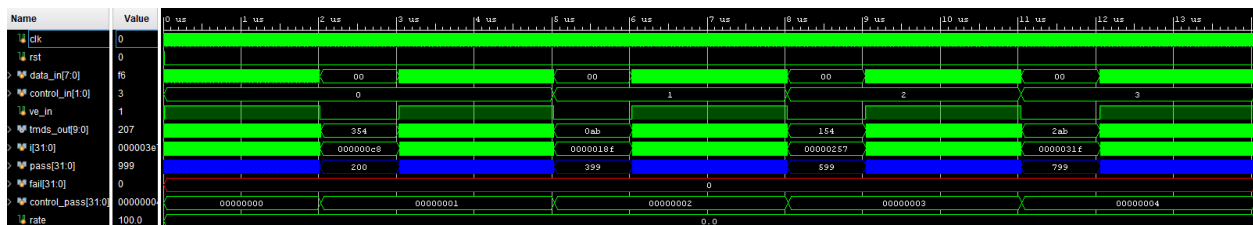
3.3 Verification

A testbench was provided in the assignment, but we also made a testbench for the transition minimization part of partC, to check if the q_m was being generated correctly.



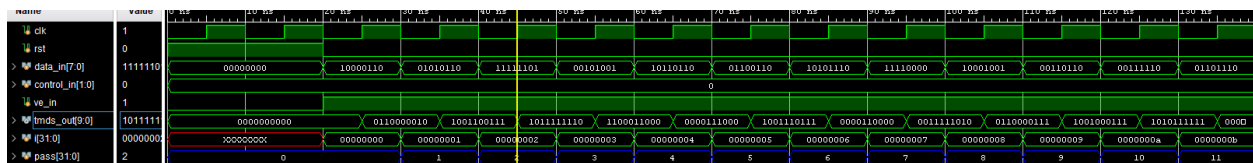
The first data_in given is the one in the assignment. The other sequences were either extreme like 00000000, 11111111 or borderline sequences with equal number of ones and zeros.

Now the full waveform, using the testbench that was given:



There is not much to add, as the waveform shows no fail occurred.

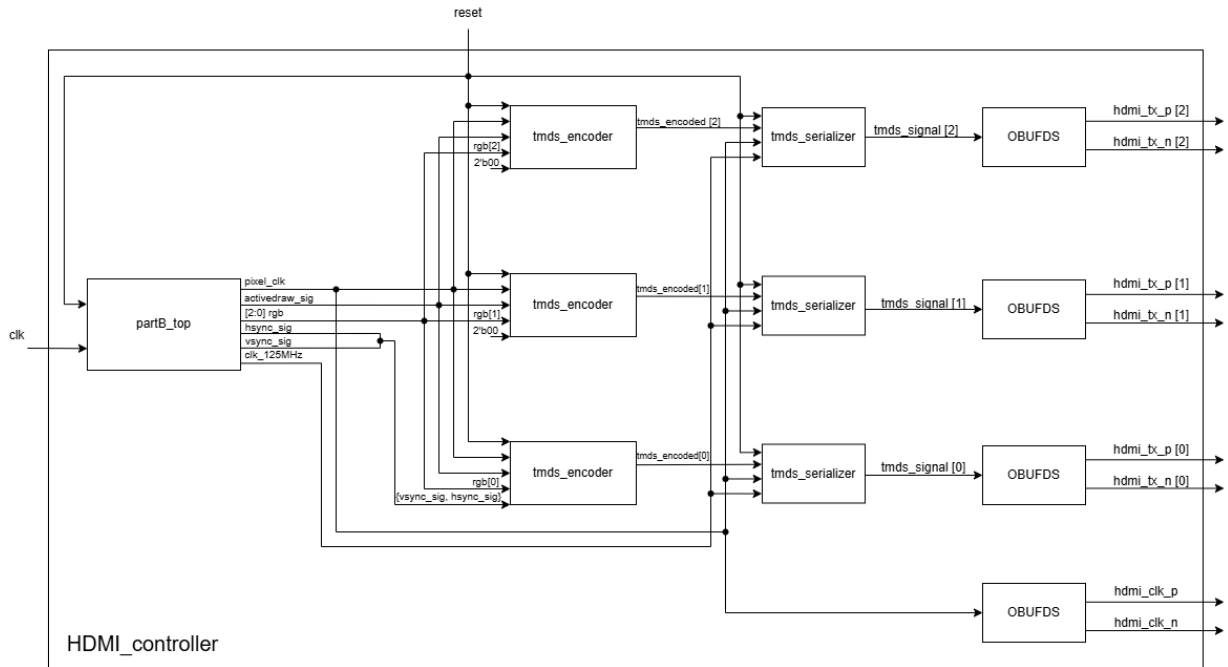
Below is a zoomed in version of the waveform showing that the corresponding q_m is generated at the positive edge of the clock.



4. Part D – HDMI Controller Integration

4.1 Introduction

Part D completes the HDMI video system by integrating all previously developed subsystems into a single, functional HDMI controller.



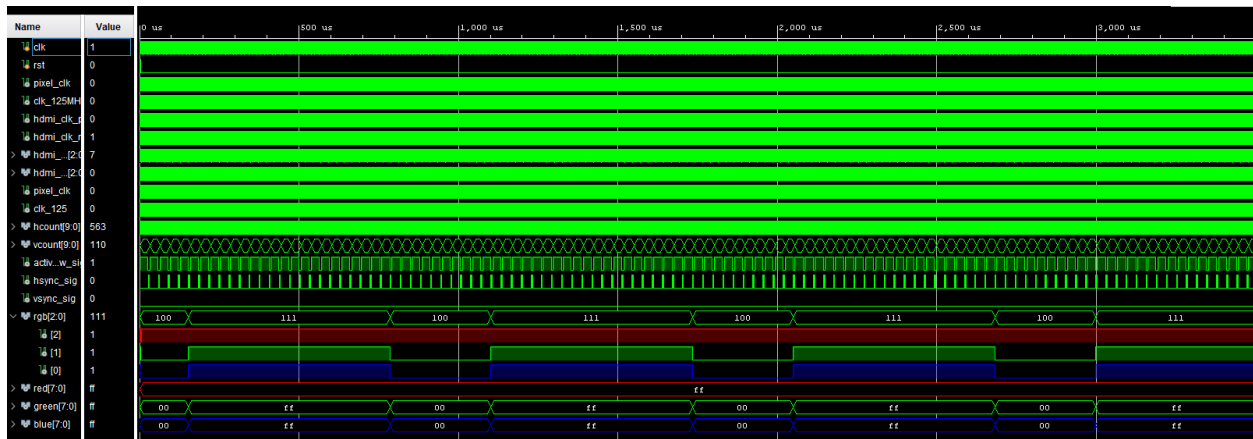
4.2 Implementation

The top-level HDMI controller module connects the video timing and VRAM logic from Part B with the TMDS encoding blocks from Part C, forming a continuous pipeline from pixel generation to HDMI output. The system operates from a 100 MHz input clock, from which the required pixel clock and higher-speed clocks are generated

Each bit from the 3-bit RGB is expanded to 8-bit color channels and passed to three TMDS encoders, one per color. The encoders generate 10-bit TMDS symbols, which are serialized using the provided TMDS serializer modules and finally driven to the HDMI connector through differential output buffers.

4.3 Verification

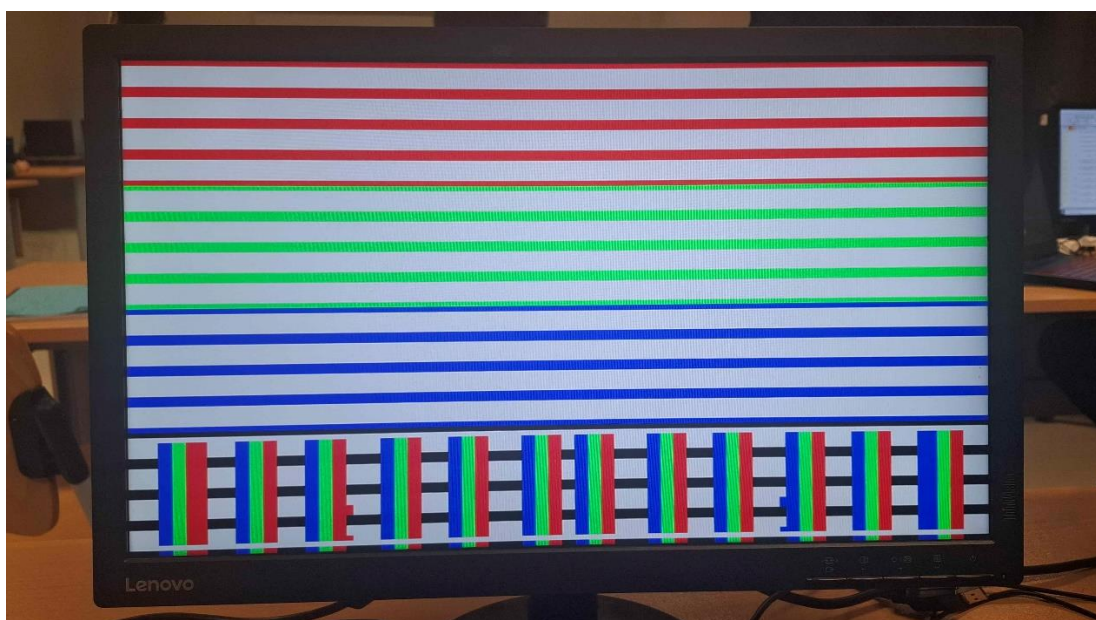
The full waveform in this part is not easily explained but:



In this snippet, where the first 4 red and white horizontal lines have been scanned, there seems to be no malfunction to any signal. Also, the red, green and blue wires are always the expanded versions of `rgb[2]`, `rgb[1]` and `rgb[0]`.

4.4 Experiment / Final Implementation

At first the picture that appeared on the screen was not exactly what we were hoping for, because we miscalculated the instantiation in the VRAM module, but the photo didn't have any problems like flickering colors or glitches. Then last Wednesday we had the correct instantiation and the picture that was generated was this:



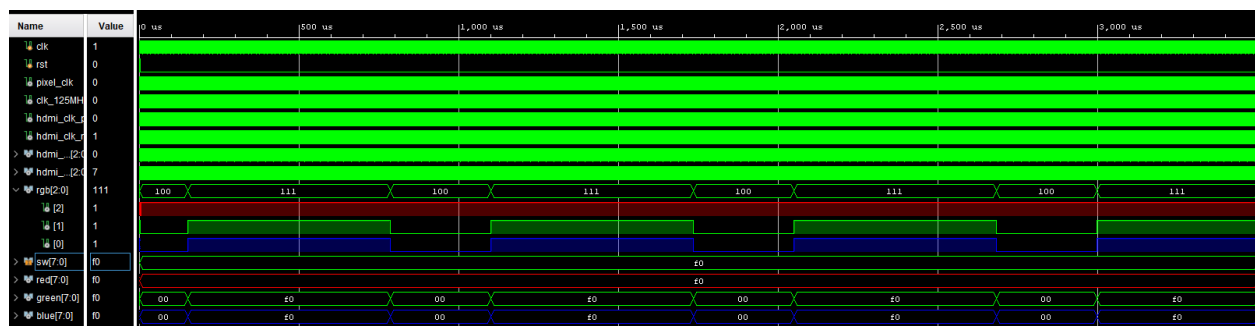
5. Part D EXTRA – HDMI Controller Integration

5.1 Introduction/Implementation

In Part D (Extra), the HDMI controller is extended with an additional 8-bit switch input from the FPGA board to enable dynamic brightness control of the displayed image. The VRAM continues to output a 3-bit RGB value, with one bit per color channel; however, instead of expanding each active color bit to a fixed 8-bit intensity, the expansion is now masked by the switch value. When a color bit is asserted, the corresponding 8-bit channel is driven by the value of the switches, while inactive color bits produce zero output. This modification allows real-time adjustment of brightness for each color without altering the overall HDMI processing pipeline. The switch inputs are integrated at the top level and constrained appropriately in the XDC file.

5.2 Verification

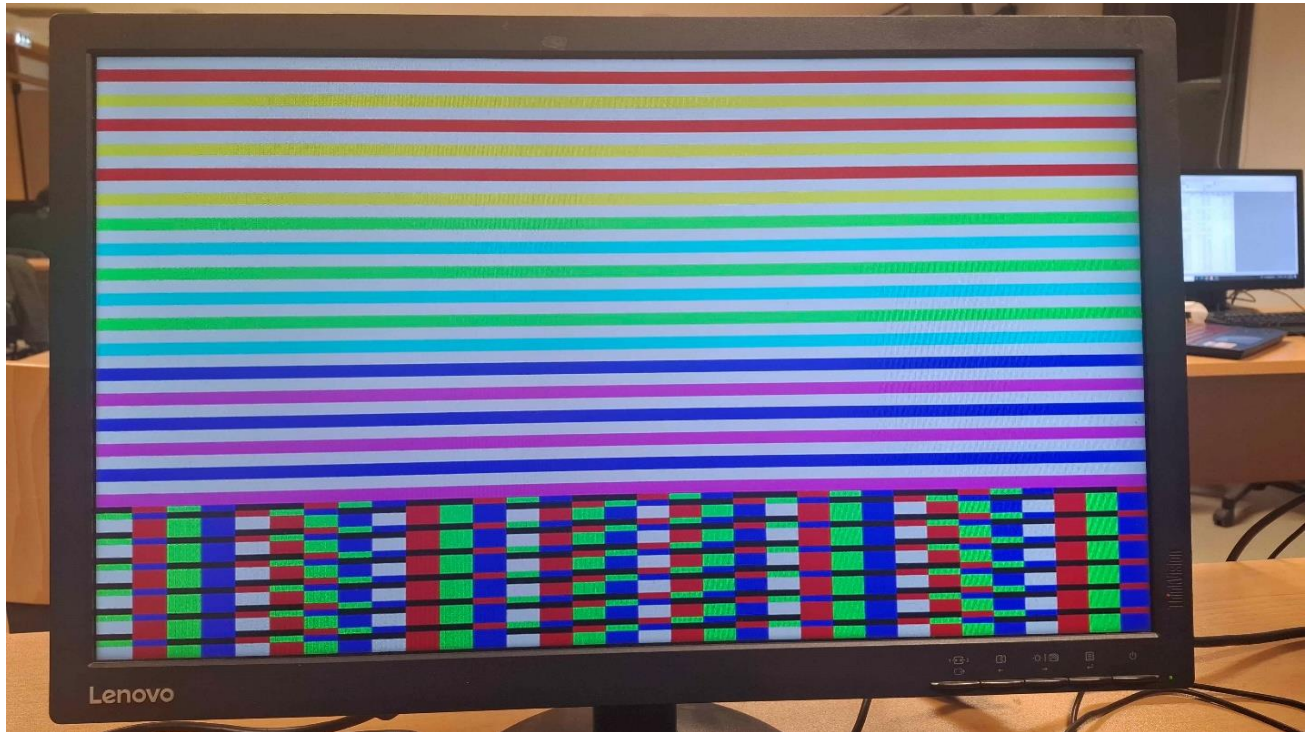
The full waveform in this PartD extra is the same:



Though the key difference is that the red, green and blue wires are still always the expanded versions of `rgb[2]`, `rgb[1]` and `rgb[0]` but with the pattern of `sw`.

5.4 Experiment / Final Implementation

Full brightness:



Low brightness:

